```
print("Factors:", factors)
Factors: (3, 5)
```

1. What you are implementing

This is a simplified demonstration of **Shor's Algorithm**.

Goal:

Factor a number $N = 15$

👉 Final answer:

- Factors = **3 and 5**

---

2. Big Picture (understand this first)

Shor's algorithm has **two parts**:

(A) Quantum Part

👉 Find the **period** r of:

$$f(x) = a^x \mod N$$

(B) Classical Part

👉 Use r to compute factors using:

$$\gcd(a^{r/2} \pm 1, N)$$

---

3. Your Classical Code (Post-processing)

```python
x = pow a  r//2  N
factor1 = gcd x - 1  N
factor2 = gcd x + 1  N
```

Example (your case)

- $N = 15$
- $a = 2$

- $r = 4$

Step:

$$x = 2^{4/2} \mod 15 = 2^2 = 4$$

Then:

- gcd(4 − 1, 15) = gcd(3, 15) = **3**
- gcd(4 + 1, 15) = gcd(5, 15) = **5**

✔️ Correct factors

---

4. Why this works (important concept)

👉 If r is period:

$$a^r \equiv 1 \mod N$$

Then:

$$a^{r/2} - 1 \quad \text{and} \quad a^{r/2} + 1$$

👉 These contain factors of N

---

5. Quantum Part (Your Circuit)

```python
qc = QuantumCircuit 4  3
```

👉 4 qubits:

- 3 → input register
- 1 → output register

Step 1: Superposition

```python
qc.   0 1 2
```

👉 Creates all inputs at once:

$|000\rangle \to \frac{1}{\sqrt{8}} \sum_{x=0}^{7} |x\rangle$

Step 2: Modular Exponentiation (Simplified)

```python
qc.   0 3
qc.   1 3
qc.   2 3
```

⚠️ Important:

👉 This is NOT real modular exponentiation
👉 It is just a teaching shortcut

Real Shor would do:

$$|x\rangle|1\rangle \to |x\rangle|a^x \mod N\rangle$$

Step 3: "Inverse QFT" (approximation)

```python
qc.  0
qc.  1
qc.  2
```

⚠️ Again simplified

👉 Real algorithm uses:

- full inverse QFT
- controlled phase rotations

Step 4: Measurement

```python
qc.        0 1 2    0 1 2
```

---

6. Your Output

```python
'000'  488  '111'  536
```

Interpretation

👉 Only two states appear:

- 000
- 111

👉 This indicates **periodic structure**

---

7. Critical Insight (this is where many misunderstand)

You might think:

> "This circuit found r = 4"

❌ Not exactly

✔️ This circuit only **suggests periodicity**

👉 You manually assumed:

```python
r = 4
```

---

8. What is Period r?

👉 Smallest number such that:

$$a^r \mod N = 1$$

Example:

x	$2^x \mod 15$
1	2

x	$2^x \mod 15$
2	4
3	8
4	1 ✅

👉 So:

- period $r = 4$

---

9. Why Quantum Helps

Classically:

- Finding period is expensive

Quantum:

- Uses **Quantum Phase Estimation** + **QFT**
- Extracts period efficiently

---

10. What your demo is missing (important critique)

Your implementation is **educational, not real Shor**

Missing:

1. True modular exponentiation circuit
2. Proper inverse QFT
3. Continued fraction step to extract r
4. Noise handling

---

11. Debugging Understanding

If factors are wrong:

Check 1:

- Is r even?

```python
if r % 2 != 0
    return None
```

Check 2:

- Does $a^{r/2} \neq -1 \mod N$?

If yes → failure case

Check 3:

- Is gcd giving trivial factors (1 or N)?

If yes → choose different a

---

12. One-line Intuition

👉 Shor's Algorithm =
"Find periodicity using quantum, convert it into factors using math"

---

13. Viva-Ready Summary

- Uses period finding
- Quantum part finds r
- Classical part computes gcd
- Exponential speedup over classical factoring

If you want deeper (recommended for you)

I can:

- ↳ show full **real Shor circuit (optimized Qiskit)**
- ↳ derive **period extraction using continued fractions**
- ↳ explain **why QFT reveals periodicity mathematically**

Tell me which direction.

Is this conversation helpful so far?